Introduction

Before you build a single n8n workflow, you have to make a decision that most tutorials skip past: where is this thing actually going to run? It feels like a throwaway setup step, but it isn't. The install method you choose determines who has access to your data, what happens when your server goes down, and whether you're paying a monthly fee for the rest of your automation career.

This matters for a reason beyond convenience. If you're using n8n professionally — automating internal tools, connecting client systems, handling anything with credentials or personal data — the hosting decision is really a data-ownership decision wearing a technical disguise. Understanding it now, before you've built twenty workflows on top of it, saves you a painful migration later.

It's also a genuinely good lens for understanding how self-hosted developer tools work in general. The same npm-vs-Docker-vs-managed-service pattern shows up everywhere: databases, CI runners, even LLM inference. Learning to reason about it once with n8n means you'll reason about it correctly the next ten times.

Problem Overview

n8n gives you three distinct ways to get it running:

1. **npm** — install the package globally on your machine and run it directly.
2. **Docker** — run n8n inside a container, with your data persisted to a mounted volume.
3. **n8n Cloud** — skip installing anything; n8n hosts and manages it for you.

All three roads lead to the exact same product once you're inside the editor — same workflows, same nodes, same behavior. What's different is everything around that: how it starts, where your data physically sits, who maintains the server, and what it costs you over time.

The goal here isn't to declare a "best" option. It's to understand what each one actually does under the hood so you can pick the one that matches what you're building.

Example

Say you're setting up n8n for the first time to automate a personal project — pulling RSS items into a spreadsheet, say. You have three starting points:

- Run `npm install n8n -g` followed by `n8n start`, and a local server boots on your machine.
- Run a `docker run` command with a `-v` flag pointing at a local folder, and a containerized server boots with your data written outside the container.
- Sign up at n8n's cloud service and get a hosted instance with a URL, no install step at all.

In every case, the first thing you see is the same: a browser window asking you to create an owner account with an email and password. That's not a coincidence — it's the same application. What differs is *where that account and its data actually live*.

Intuition

Here's how an experienced engineer thinks about this before writing a single command.

Any self-hostable tool has to answer three questions: where does the process run, where does the process store its state, and who's responsible for keeping the process alive. Once you separate those three concerns, the three install methods stop looking like arbitrary options and start looking like three different answers to the same set of questions.

- **npm** answers "run it directly on my machine's Node runtime, store state in my home directory, and I'm responsible for keeping it running."
- **Docker** answers the same three questions, but wraps the runtime in an isolated, disposable container — while explicitly separating the *process* (throwaway, container) from the *state* (persistent, a mounted volume on the host).
- **Cloud** answers all three questions by handing them to n8n entirely: their server, their storage, their uptime responsibility. You trade control for convenience.

Once you see it this way, the "which one should I pick" question becomes a straightforward tradeoff between control and maintenance burden — not a random preference.

Brute Force Solution — npm Install

Idea: Get n8n running with the least ceremony possible. Install the package globally, run the start command, done.

Algorithm: 1. Confirm Node.js is installed. 2. `npm install n8n -g` 3. `n8n start` 4. Open the local URL in a browser.

Advantages: - Fastest possible path from zero to a running instance. - No container concepts to learn. - Direct access to logs and the filesystem.

Disadvantages: - Tightly coupled to your machine's Node version and global package state. - Upgrading or reinstalling can be messier than swapping a container image. - No built-in isolation — if something else on your machine is misbehaving with ports or dependencies, it can interfere.

Complexity: Setup time is O(1) — a couple of commands. Storage is a single directory on disk (`~/.n8n`), so no meaningful space overhead beyond the app itself.

Optimal Solution — Docker with a Mounted Volume

This is the version worth actually explaining in depth, because it's where the "aha" of the lesson lives: the container is disposable, the data is not.

When you run n8n in Docker with a `-v` flag, you're telling Docker to mount a folder from your host machine into the container at the path where n8n writes its data. That single flag is what separates "container" from "container's contents."

Without `-v`	With `-v`
Data lives inside the container's writable layer	Data lives in a folder on your host machine
Deleting/rebuilding the container deletes your workflows	Deleting/rebuilding the container leaves your workflows untouched
Upgrading the image wipes your state	Upgrading the image just swaps the runtime; data persists

Think of the container as a fresh, disposable copy of the n8n application every time it starts — and the mounted volume as the one thing that survives across every rebuild. This is the same pattern used by databases in Docker (mount a volume at the data directory) and it's worth internalizing once, because it applies far beyond n8n.

Steps: 1. Pull the official n8n Docker image. 2. Run it with a volume flag pointing your host folder at n8n's data directory. 3. n8n boots inside the container, listening on its default port. 4. Your browser connects to that port; the owner account and all workflow data get written straight to your mounted host folder.

Python Code

There's no Python involved in installing n8n itself — but if you're scripting the setup (say, for a reproducible dev environment), here's a small wrapper that checks prerequisites and launches the right install path:

```
import shutil
import subprocess
import sys


def check_command(name: str) -> bool:
    """Return True if `name` is available on PATH."""
    return shutil.which(name) is not None


def install_with_npm() -> None:
    subprocess.run(["npm", "install", "n8n", "-g"], check=True)
    subprocess.run(["n8n", "start"], check=True)


def install_with_docker(data_dir: str) -> None:
    subprocess.run(
        [
            "docker", "run", "-it",
            "-p", "5678:5678",
            "-v", f"{data_dir}:/home/node/.n8n",
            "n8nio/n8n",
        ],
        check=True,
    )


def main() -> None:
    if check_command("docker"):
        install_with_docker(data_dir="./n8n_data")
    elif check_command("npm"):
        install_with_npm()
    else:
        sys.exit("Neither Docker nor npm found — install one before continuing.")


if __name__ == "__main__":
    main()
```

Code Walkthrough

- `check_command` uses `shutil.which`, the standard way to check if a binary is on `PATH` — no need for a manual `os.system("which ...")` call.
- `install_with_npm` mirrors the two-command manual flow: install the package globally, then start it. `check=True` makes the subprocess call raise if either step fails, instead of silently continuing.

- `install_with_docker` reproduces the volume-mount pattern from the lesson: `-p 5678:5678` exposes n8n's default port to your host, and `-v {data_dir}:/home/node/.n8n` is the line that makes your workflows persist outside the container.
- `main` prefers Docker when available, since it gives you both isolation and persistence with one command, falling back to npm otherwise.

Dry Run

Say you run this script on a machine with Docker installed and `data_dir="./n8n_data"`.

1. `check_command("docker")` returns `True`.
2. `install_with_docker("./n8n_data")` builds the Docker command and runs it.
3. Docker pulls the `n8nio/n8n` image if it isn't cached locally.
4. The container starts, binds port 5678, and mounts `./n8n_data` to `/home/node/.n8n` inside the container.
5. n8n boots, writes its initial state (config, database file) to that mounted path.
6. You open `localhost:5678`, see the owner-account creation screen, and every workflow you build from here gets written to `./n8n_data` on your actual machine — visible even after the container is deleted.

Complexity Analysis

There's no algorithmic complexity here in the classic sense — this is a systems/setup problem, not a computational one. The more useful lens:

- **Time to running instance:** O(1) for all three methods — each is a fixed, small number of steps regardless of "input size."
- **Maintenance complexity over time:** npm and Docker scale linearly with how many self-hosted instances you manage (you patch, back up, and monitor each one). Cloud is effectively O(0) on your end — n8n absorbs that cost, which is exactly what you're paying for.

Alternative Solutions

n8n Cloud is the real alternative worth comparing directly, since it's not just "another install method" — it removes the install step entirely.

	Self-hosted (npm/Docker)	n8n Cloud
Setup	You run it	Nothing to run

	Self-hosted (npm/Docker)	n8n Cloud
Data location	Your machine/server	n8n's servers
Cost	Free (your infra)	Monthly subscription
Maintenance	You patch/back up	Handled for you
Best for	Full control, sensitive data, cost-sensitive projects	Zero-maintenance, fast onboarding, teams without ops capacity

Neither is objectively better — it depends on whether you're optimizing for control or convenience.

Edge Cases

- **Port already in use:** if you try to start a second local n8n instance while one is already running, it silently fails to bind to the default port (5678). Nothing tells you clearly what happened — check for an existing process first.
- **No Docker volume mounted:** running the container without `-v` means every workflow disappears the moment you delete or rebuild the container.
- **Webhook nodes on an unreachable instance:** if you're testing a webhook trigger locally without exposing your instance to the internet (via a tunnel or public deployment), the trigger simply never fires — it looks like a broken workflow, but it's a networking issue.
- **Environment variables set after boot:** n8n reads `.env` values once at startup. Changing them while the process is running has no effect until you restart.
- **Wrong home directory permissions:** if `~/.n8n` (or your mounted volume path) isn't writable, the app can fail to persist state without an obvious error pointing at permissions.

Common Mistakes

1. **Running Docker without a mounted volume**, then losing all workflow data on the next container rebuild.
2. **Starting a second local instance** without noticing the first one already owns port 5678, leading to confusing silent failures.
3. **Assuming Cloud and self-hosted are functionally different products** — they're the same app; only the deployment model changes.
4. **Testing webhooks on a local instance with no public exposure**, then debugging the workflow logic when the real issue is that n8n was never reachable from the internet.
5. **Editing `.env` values and expecting live effect** — forgetting these are read once at boot.
6. **Treating self-hosting as "set and forget"** — underestimating the ongoing patching/backup responsibility that comes with owning the server.

Interview Questions

- What's the practical difference between a container's filesystem and a mounted volume, and why does that matter for stateful applications?
- How would you design a Docker setup for an app that needs persistent state across container rebuilds?
- What's the tradeoff between self-hosting a service versus using a managed/cloud version of the same tool?
- Why do environment variables typically get read once at process startup rather than watched for changes?
- How would you debug a webhook or callback that "isn't firing" when the receiving service is running locally?

Similar Problems

- **Setting up a self-hosted database with Docker** (e.g., Postgres) — same volume-mounting pattern for persistent state.
- **Self-hosted vs. managed CI runners** — the same control-vs-convenience tradeoff shows up in GitHub Actions self-hosted runners vs. GitHub-hosted ones.
- **Local dev server vs. hosted preview environments** — same "who owns the infra" decision in a different context.
- **Any tool with a `.env`-based configuration system** — the "read once at boot" behavior generalizes broadly.

Key Takeaways

- npm, Docker, and Cloud all run the identical n8n application — the difference is entirely in deployment, not functionality.
- In Docker, the container is disposable; the mounted volume is what actually matters for data persistence.
- Self-hosting trades maintenance burden for control and zero subscription cost; Cloud trades money for zero maintenance.
- Environment variables are read once at startup — treat them like a dial you set before pressing start, not a live setting.
- Most beginner pain comes from two things: port conflicts between multiple local instances, and webhook triggers that silently never fire because the instance isn't reachable from the internet.

Watch the Video

This entire walkthrough is demonstrated live in the video — watching the actual install commands run and the owner-account screen appear across all three methods makes the differences click faster than reading alone. {LINK}

About the Series

This is Lesson 2 of the n8n series inside **Fun with Learning Technology** — a series built around the idea that following steps isn't the same as understanding why they work. Each lesson takes one piece of a real developer tool and breaks down the reasoning behind it, not just the commands to type. Lesson 3 picks up right where this one ends: opening n8n and building your first real workflow.

Call To Action

If this helped the install decision finally make sense, do three things: like the video on YouTube so more developers see it, subscribe so you catch Lesson 3 when it drops, and subscribe to the Substack for the written breakdowns like this one. Drop a comment if something felt unclear or if there's a topic you want covered next — that feedback directly shapes what gets made.

Quick Review

When installing n8n, what's the first fork in the road?

Self-hosted (npm/Docker) vs n8n Cloud — decides if you pay monthly forever or run free on your own machine.

npm vs Docker: which needs more upfront setup?

npm installs faster with less isolation; Docker takes a bit more setup but keeps n8n and dependencies contained.

What's the real tradeoff between self-hosted and Cloud?

Self-hosted = free but you own uptime, updates, backups; Cloud = paid but zero maintenance.

What happens the very first time n8n starts?

It spins up a local server (default port 5678) and prompts you to create the owner account before any workflow access.

Why do environment variables matter on first run?

They set core rules — port, timezone, encryption key, webhook URL — before the owner account or workflows exist.

Where does n8n actually store your data by default?

In a local SQLite file under the user data folder, unless you configure Postgres/MySQL or a Docker volume.
